자바 컬렉션 프레임워크(Collection Framework) 핵심 정리

1. 컬렉션 프레임워크의 개념

- **정의**: 다수의 데이터를 쉽고 효과적으로 처리할 수 있는 표준화된 방법을 제공하는 클래스들의 집합입니다.
- **역할**: 데이터를 저장하는 자료구조와 데이터를 처리하는 알고리즘을 구조화하여 클래스로 구현해 놓은 것입니다.
- **특징**: 자바의 인터페이스(Interface)를 사용하여 구현되며, 타입 안정성을 위해 제네릭(Generic) 형태로 제공됩니다.

2. 주요 인터페이스 구조 및 계층 관계

- **Collection<E>**: List와 Set의 공통된 요소를 정의하는 최상위 인터페이스입니다. (상위에 Iterable<E> 존재)
    - **List<E>**: 순서가 있는 데이터의 집합이며, 데이터의 중복을 허용합니다. (구현 클래스: ArrayList, LinkedList, Vector, Stack 등)
    - **Set<E>**: 순서가 없는 데이터의 집합이며, 데이터의 중복을 허용하지 않습니다. (구현 클래스: HashSet, TreeSet 등)
- **Map<K, V>**: 키(Key)와 값(Value)의 쌍으로 데이터를 관리하므로 Collection 인터페이스와 별도로 독립된 계층을 형성합니다.
    - 키(Key)는 중복 불가, 값(Value)은 중복 허용 (구현 클래스: HashMap, TreeMap, Hashtable, Properties 등)

3. 핵심 인터페이스 3대 비교

인터페이스	순서(Index)	중복 허용 여부	주요 구현 클래스	특징 및 요약
List<E>	있음 (유지)	허용 (값 중복 가능)	ArrayList, LinkedList, Vector, Stack	순서대로 데이터를 나열해 저장하며, 인덱스를 통한 조회가 빠름
Set<E>	없음	불가 (중복 자동 제거)	HashSet, TreeSet	집합 개념으로, 중복 값을 허용하지 않는 고유 데이터 저장에 사용
Map<K, V>	없음	키: 중복 불가 / 값: 중복 허용	HashMap, TreeMap, Hashtable, Properties	키를 통해 값을 검색하는 딕셔너리 구조

클래스 선택 권장사항: 과거 레거시 클래스인 Vector, Hashtable보다는 동기화 부담이 적고 성능이 뛰어난 ArrayList, HashMap 사용을 권장합니다.

4. Collection<E> 인터페이스 주요 메서드

- `boolean add(E e)`: 요소 추가
- `boolean remove(Object o)`: 특정 요소 제거
- `void clear()`: 모든 요소 제거
- `int size()`: 저장된 요소 개수 반환
- `boolean isEmpty()`: 비어있는지 확인
- `boolean contains(Object o)`: 특정 객체 포함 여부 확인
- `Iterator<E> iterator()`: 요소 순회를 위한 반복자 반환
- `Object[] toArray()`: 배열로 변환